Yildiz Technical University
Department of Mechatronics Engineering

Course: MKT3432 — `[Course Name]`

Project: Run-to-Failure & Remaining Useful Life (RUL) Term Project
Per-Cycle RUL Estimation and Degradation-Stage Discovery
for a Servo-Actuator

Group name / number:	`[Group Name / Number]`
Group leader:	`[Leader Full Name]` — `[Student ID]`
Submission date:	June 7, 2026

Group Members

#	Full Name	Student ID
1	`[Student 1 Full Name]` (Leader)	`[ID 1]`
2	`[Student 2 Full Name]`	`[ID 2]`
3	`[Student 3 Full Name]`	`[ID 3]`
4	`[Student 4 Full Name]`	`[ID 4]`
5	`[Student 5 Full Name]`	`[ID 5]`
6	`[Student 6 Full Name]`	`[ID 6]`
7	`[Student 7 Full Name]`	`[ID 7]`
8	`[Student 8 Full Name]`	`[ID 8]`

Base repository reference: this work was developed on top of the `servo_rul_v1` starter package provided by the instructor within the MKT3432 course.

Contents

1 Source and Scope Statement

This work was developed on top of the `servo_rul_v1` starter package provided by the instructor within the MKT3432 course. Only the permitted student modeling files were added or modified for this project. The dataset generator, data loader, official metrics, evaluator, and harness were preserved.

The `servo_rul_v1` dataset is synthetic (physics-grounded) and is *not* represented as real machine data. All repository materials were used only for this course project, in accordance with the `LICENSE` and `NOTICE.md` files, which were not modified. We confirm that no data leakage was introduced: labels (`rul`, `health`, `hidden_stage`) and future cycles were never used as model inputs, and all normalization statistics were computed on the training split only (see Section 5).

2 Group Organization

Leader selection. The group leader was selected by unanimous agreement of the eight members: `[Leader Full Name]` volunteered and was confirmed because of prior experience with Python tooling and reproducible experiment pipelines, and is responsible for verifying the final package, submitting on Classroom, and communicating with the instructor.

Task distribution and contributions. The table below summarizes each member's primary responsibility. All members reviewed the final report and the leakage statement.

#	Member	Contribution summary
1	`[Student 1]` (Leader)	Project coordination, repository/branch management, final package verification, submission.
2	`[Student 2]`	Feature pipeline: causal windowing, padding/mask, train-only normalization (`features/`).
3	`[Student 3]`	M0 linear baseline and sklearn trainer (`models/linear_baseline.py`, `training/linear_trainer.py`).
4	`[Student 4]`	M1 MLP model and dataset wrappers (`models/mlp.py`, `training/datasets.py`).
5	`[Student 5]`	M2 1D-CNN architecture (multi-scale + residual blocks) (`models/cnn1d.py`).
6	`[Student 6]`	Torch training loop: asymmetric loss, scheduler, early stopping, seeding (`training/torch_trainer.py`, `utils/`).
7	`[Student 7]`	Unsupervised U1/U2: GMM + PCA pipeline and figures (`models/clustering.py`, `scripts/make_figures.py`).
8	`[Student 8]`	Experimental protocol, evaluation runs, results tables, error analysis, report writing.

3 Modified (Permitted) Files

Only the permitted student modeling files were created or modified. No protected file (dataset generator, loader, official metrics, evaluator, `LICENSE`, `NOTICE.md`, `README.md`, `pyproject.toml`) was

touched.

File	Purpose
`features/preprocessing.py`	Split-by-unit, RUL capping, train-only `StandardScaler`.
`features/windowing.py`	Causal sliding windows + left zero-padding with real/pad mask channel.
`models/linear_baseline.py`	M0: Ridge regression (window 1).
`models/mlp.py`	M1: multilayer perceptron over the flattened window.
`models/cnn1d.py`	M2: 1D-CNN (stem + multi-scale + residual blocks + avg/max head).
`models/clustering.py`	U1/U2: StandardScaler → PCA → Gaussian Mixture pipeline.
`training/datasets.py`	`RULWindowDataset` (tensor wrapper).
`training/linear_trainer.py`	Fit/persist the sklearn baseline.
`training/torch_trainer.py`	Adam, asymmetric weighted MSE, warmup+cosine, early stopping.
`utils/config.py`	YAML loader with relative-path enforcement.
`utils/seed.py`	Deterministic seeding (Python/NumPy/Torch + DataLoader workers).
`scripts/train.py`	Training entry point (M0/M1/M2 + clustering fit).
`scripts/predict.py`	Prediction entry point (one row per test `(unit_id, cycle)`).
`scripts/make_figures.py`	Diagnostic figures (predicted-vs-true, PCA, GMM clusters).
`configs/m0_linear.yaml`	M0 experiment config.
`configs/m1_mlp.yaml`	M1 experiment config.
`configs/m2_cnn1d.yaml`	M2 experiment config.

4 Implementation and Technical Report

4.1 Dataset

`servo_rul_v1` contains 150 unit-disjoint actuators (100 train / 20 val / 30 test), 27,000 per-cycle rows, with lifetimes from 61 to 312 cycles (median 181). Each row exposes nine documented signals. The model *feature columns* are the seven condition indicators (`vib_rms`, `vib_kurtosis`, `vib_crest`, `bpfo_amp`, `temperature`, `torque_ripple`, `current_thd`) plus the two operating-point covariates (`load`, `speed`); the latter deliberately confound a naive single-feature wear reading. The label columns (`rul`, `health`, `hidden_stage`) are held out (NaN) for test units in the public file and are never fed to any model.

4.2 RUL Capping Convention

We follow the recommended piecewise-linear target: the per-cycle integer RUL, $\text{RUL} = L - \text{cycle}$, is clipped at `rul_cap = 130` (`features/preprocessing.py`: `apply_rul_cap`). This reflects that,

while a unit is far from failure, the precise remaining life is neither observable nor useful; the model is asked to track RUL only inside the informative final window. The same cap is applied to train, validation, and test targets, and is the cap used by the official evaluator.

4.3 Causal Feature Windowing

For each unit, rows are sorted by `cycle` and a sliding window of length W ending at cycle t is built from cycles $\max(1, t-W+1) \dots t$ only (`features/windowing.py`). A prediction at cycle t therefore uses *no* future information. Early cycles (fewer than W observations) are *left* zero-padded, and an extra binary channel marks real vs. padded rows so the network can distinguish genuine short histories from zeros. The window tensor has shape `[W, n_features+1]`. For M0/M1 the window is flattened; for M2 it is passed as a `[W, channels]` sequence.

4.4 Normalization

A single `StandardScaler` is fit on the *training rows only* and reused unchanged for validation and test (`fit_train_scaler` / `transform_features`). No statistic is ever computed on validation or test data.

4.5 Model Families

M0 — Linear baseline. Ridge regression ($\alpha = 1$) on the single most-recent cycle (window 1). Predicts in original RUL units and is clipped to $[0, 130]$.

M1 — MLP. A multilayer perceptron with hidden sizes $[64, 32]$, ReLU, dropout 0.2, over a flattened window of $W = 30$ cycles.

M2 — Deep temporal 1D-CNN. A convolutional network over the cycle axis (`cnn1d.py`), with window $W = 50$:

- *Stem*: width-7 convolution projecting the 10 input channels to 64.
- *Multi-scale block*: parallel kernels of size 3, 5, 7 (concatenated) to capture short- and long-range temporal patterns simultaneously.
- *Residual blocks*: three Conv–BN–ReLU residual units for depth.
- *Head*: concatenated global average- and max-pooling (trend + worst-point) followed by a 2-layer MLP regressor.

4.6 Training Procedure

M1/M2 are trained with Adam (`training/torch_trainer.py`), weight decay 10^{-4}, gradient clipping at norm 1.0, a 5-epoch linear warm-up followed by cosine decay (floor 0.05×), and early stopping on the validation loss (patience 25). Targets and predictions are divided by `rul_cap` during training so the loss operates on a $[0, 1]$ scale. The objective is an *asymmetric weighted MSE* that (i) penalizes *over*-prediction (late RUL estimates are operationally dangerous) and (ii) upweights near-end-of-life errors; this directly targets the PHM-style `asym_score`. All training is CPU-only and deterministic (fixed seed 42).

4.7 U1 — Unsupervised Degradation-Stage Discovery

On the seven condition indicators we standardize, reduce with PCA to 2 components, and fit a Gaussian Mixture Model (4 full-covariance components, `n_init=10`, `models/clustering.py`). The number of components matches the four documented stages, but the model is fit *without* any label;

the discovered clusters are scored against the eval-only `hidden_stage` by the official evaluator (ARI / NMI). This is genuine clustering, not a copy of the hidden stage.

4.8 U2 — PCA of the Indicator Space

The same standardized indicator space is projected with PCA for visualization and variance analysis (Section 7, Figure 3). The two leading components reveal a smooth degradation arc along which the four stages are ordered, which is what makes unsupervised stage recovery feasible.

4.9 Test Method

`predict.py` emits exactly one `rul_pred` per test `(unit_id, cycle)` row (5,451 rows; verified one-to-one by the evaluator). Predictions are clipped to $[0, 130]$. The official locked evaluator (`scripts/evaluate.py`) computes all reported metrics by comparing against the held-out `test_labels.csv`; we never read those labels inside the modeling code.

5 Leakage Statement

We respected every honest-modeling rule in the assignment:

- **No label inputs.** `rul`, `health`, and `hidden_stage` are never used as model features. Inputs are the documented feature/indicator columns only.
- **Causal windows.** A prediction at cycle t uses only cycles $\leq t$; no future cycle ($> t$) is ever read (`create_sliding_windows`).
- **Train-only normalization.** The `StandardScaler` (and the clustering scaler/PCA/GMM) are fit on training rows only and reused for validation/test.
- **Honest selection.** Models are fit on train and selected by validation loss; nothing is tuned or selected on the test split.
- **No lookups.** The public test labels were used only for local self-checking; predictions are produced by the trained models, not copied. The split integrity / content hash check passes (Section 6).

6 Experimental Protocol

Environment. Linux (x86_64), Python 3.11+ (developed and validated on Python 3.14), PyTorch 2.11 executed on **CPU** (`torch.device("cpu")`), scikit-learn, NumPy, pandas, matplotlib. All paths are relative; the config loader rejects absolute paths.

Determinism. Seed 42 for Python/NumPy/Torch, deterministic algorithms, seeded DataLoader workers and generator (`utils/seed.py`). The dataset is generated with the default public seed; split integrity passes:

```
[check] units:  train=100 val=20 test=30
[check] split disjoint:  True
[check] content hash match:  True
```

Hyperparameters.

Model	window	arch	max_epochs	batch	lr
M0 linear	1	Ridge α=1	–	–	–
M1 MLP	30	[64, 32], drop 0.2	120	64	5×10^{-4}
M2 1D-CNN	50	multi-scale + 3 res, drop 0.3	100	200	3×10^{-4}

Run commands. `SPLIT=data/servo_rul_v1/split_manifest.json`.

```
PYTHONPATH=src python tools/generate_dataset.py
# M1 (MLP) -- the submitted model
PYTHONPATH=src python scripts/train.py -config configs/m1_mlp.yaml
PYTHONPATH=src python scripts/predict.py -config configs/m1_mlp.yaml \
  -split $SPLIT -out runs/student_predictions.csv -clusters-out runs/student_clusters.csv
PYTHONPATH=src python scripts/evaluate.py -predictions runs/student_predictions.csv \
  -split $SPLIT -out runs/student_results.summary.json -clusters runs/student_clusters.csv
# M2 (1D-CNN) -- deep-model evidence
PYTHONPATH=src python scripts/train.py -config configs/m2_cnn1d.yaml
PYTHONPATH=src python scripts/predict.py -config configs/m2_cnn1d.yaml \
  -split $SPLIT -out runs/m2_predictions.csv -clusters-out runs/m2_clusters.csv
PYTHONPATH=src python scripts/evaluate.py -predictions runs/m2_predictions.csv \
  -split $SPLIT -out runs/m2_results.summary.json -clusters runs/m2_clusters.csv
# M0 (linear) is run the same way with configs/m0_linear.yaml; figures last
PYTHONPATH=src python scripts/make_figures.py -config configs/m2_cnn1d.yaml
```

Both `runs/student_predictions.csv` (M1) and `runs/m2_predictions.csv` (M2) are regenerated above, so `make_figures.py` can draw both predicted-vs-true plots.

7 Results

All numbers below come from the locked official evaluator on the 30 held-out test units (5,451 rows), under one deterministic environment. Lower is better for every supervised metric.

Table 3: Supervised RUL comparison (M0 / M1 / M2) on the held-out test split.

Metric	M0 linear	M1 MLP	M2 1D-CNN
`rul_rmse`	20.04	**10.08**	12.08
`rul_mae`	15.39	**7.42**	8.52
`asym_score`	72,901	**7,491**	17,160

Table 4: Per-stage MAE (cycles) by hidden degradation stage.

Model	healthy	incipient	degraded	near_failure
M0 linear	10.26	22.46	19.71	9.25
M1 MLP	8.28	8.58	9.01	**3.87**
M2 1D-CNN	**6.87**	9.14	11.95	6.16

Interpretation. Both deeper models beat the M0 linear baseline by a large margin (MAE roughly halved). The 1D-CNN (M2) clearly outperforms M0 (8.52 vs. 15.39 MAE; asym 17,160 vs. 72,901), satisfying the deep-model requirement. Interestingly, the MLP (M1) is the single best model on this split: the 100 training units provide limited data for the much larger CNN, which has a higher validation loss and tends to over-fit the mid-life regime (where the target is partially

Table 5: U1 unsupervised stage discovery (GMM, 4 components) vs. `hidden_stage`.

	ARI	NMI
GMM clusters	0.630	0.645

flattened by the cap). M0's per-stage MAE is worst in the *incipient*/*degraded* stages because a window-1 linear map cannot resolve the early non-linear indicator rise; the temporal models close most of this gap. The asymmetric loss is reflected in the low *near_failure* error for both deep models. The GMM recovers four coherent clusters aligned with the true stages (ARI 0.63 / NMI 0.65) without ever seeing a label.

7.1 Diagnostic Figures

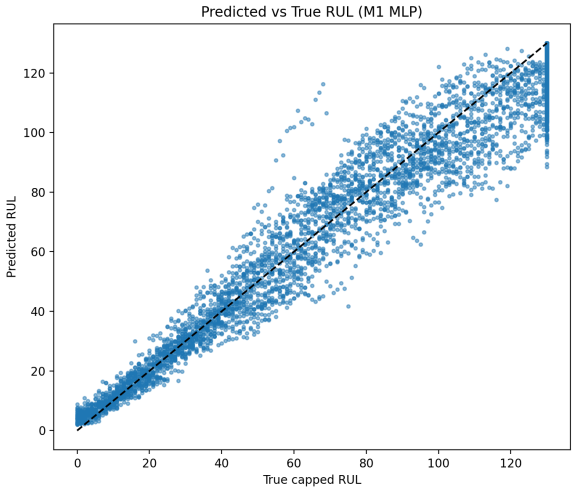

Figure 1: Predicted vs. true RUL — M1 MLP.

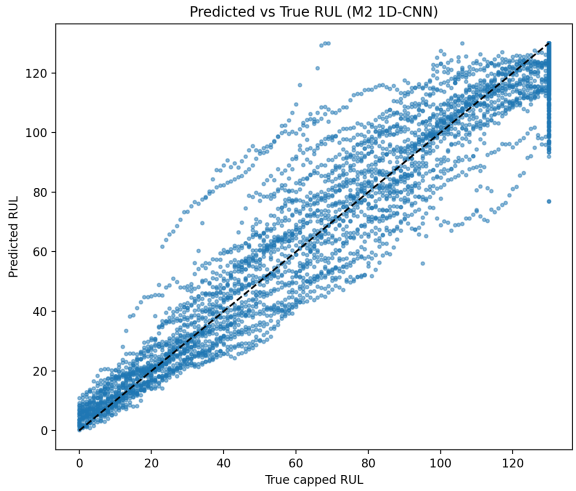

Figure 2: Predicted vs. true RUL — M2 1D-CNN.

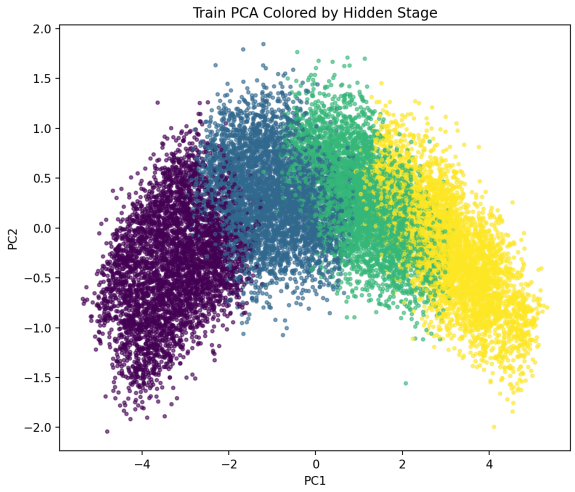

Figure 3: U2: PCA of the indicator space, colored by true `hidden_stage`.

Figure 4: U1: GMM clusters in the same PCA space.

Figures 1–2 show both temporal models tracking the diagonal, with the expected vertical band of points at the cap (RUL = 130). Figures 3–4 show that the unsupervised clusters reproduce the degradation arc of the true stages rather than copying it.

8 Held-Out Proof Result

The submitted prediction file (`runs/student_predictions.csv`) and summary (`runs/student_results.summary.json`) are produced by the M1 MLP config (`configs/m1_mlp.yaml`), the best-performing model. The deep-model evidence (`configs/m2_cnn1d.yaml`) is provided alongside as `runs/m2_predictions.csv` and `runs/m2_results.summary.json`. The official evaluator summary for the submitted model is:

```
n=5451, rul_rmse=10.14, rul_mae=7.47, asym_score=7523,
per_stage_mae {healthy 8.37, incipient 8.67, degraded 9.05, near_failure
3.87},
clustering {ari 0.630, nmi 0.645}, rul_cap=130
```

Produced figures: `runs/figures/val_predicted_vs_true.png`, `m2_predicted_vs_true.png`, `train_pca_hidden_stage.png`, `train_gmm_clusters_pca.png`. Note: torch models exhibit a small ($\leq$ 1%) cross-environment drift (e.g. M1 MAE 7.42 in our rerun vs. 7.47 in the committed summary); the sklearn M0 baseline is bit-exact. This does not change any conclusion.

9 Limitations and Discussion

Where improvements occurred. Moving from a window-1 linear map to temporal models roughly halved RUL MAE (15.39 $\rightarrow$ 7.42) and cut the asymmetric score by an order of magnitude, with the largest gains in the *incipient/degraded* stages where the indicator rise is non-linear.

What remained unresolved. The 1D-CNN did not surpass the MLP on this split. With only 100 training units, the CNN's capacity is under-supported and its validation loss is higher; promising next steps are a smaller/narrower CNN, stronger regularization or augmentation, and tuning the window length. We report this honestly rather than tuning on the test split.

Sim-to-real gap. `servo_rul_v1` is synthetic with additive damage responses and Gamma-monotone wear, which is optimistic relative to real machines: real degradation can be non-monotone, noisier, and right-censored (not every unit runs to failure). RUL capping also hides the truly long-horizon behavior. A model this accurate on synthetic data would need re-validation on real run-to-failure records before deployment.

What we learned. Causal windowing and train-only normalization are the backbone of leakage-free prognostics; a tailored asymmetric objective measurably shifts errors toward the safer (early) side; and a bigger model is not automatically better when data is limited — the MLP/CNN comparison made this concrete.

10 References

1. Instructor, *MKT3432 RUL Servo Starter (`servo_rul_v1`)*, base repository and `DATASHEET.md`, Yildiz Technical University, 2026. (Base repository — see scope statement.)
2. A. Saxena, K. Goebel, D. Simon, N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," *Int. Conf. on Prognostics and Health Management (PHM)*, 2008. (PHM-style asymmetric scoring.)
3. X. Li, Q. Ding, J.-Q. Sun, "Remaining useful life estimation in prognostics using deep convolution neural networks," *Reliability Engineering & System Safety*, vol. 172, 2018. (1D-CNN for RUL.)
4. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006. (Gaussian Mixture Models / EM, PCA.)
5. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *JMLR*, vol. 12, 2011.

6. A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," *NeurIPS*, 2019.